

PROJECT REPORT

EDUGENIE

Google Gemini Powered Learning Assistant

An AI-Powered Educational Web Application

University	University of Madras (Madras University)
College	Quaid-e-Millath Government College for Women (Autonomous)
College Code	1110005
Student Name	Mary.R
Project	EduGenie – Google Gemini Powered Learning Assistant
Academic Year	2026

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Use-Case Scenarios
6. System Architecture
7. Prerequisites and Technologies
8. Project Workflow
9. Folder Architecture
10. Core Functional Modules
11. Backend API with FastAPI
12. Frontend Development
13. Coding Implementation
14. Sample Input and Output
15. Functional Testing
16. Importance of EduGenie
17. Advantages
18. Limitations
19. Future Enhancements
20. Conclusion
21. Project Submission Details

1. Introduction

EduGenie is an AI-powered learning assistant designed to support students through an easy-to-use web interface. The system combines a FastAPI backend with educational modules and Google Gemini-based AI processing. It can answer questions, explain concepts in simple language, generate quizzes, summarize long passages, and provide structured learning recommendations.

The project demonstrates practical application of artificial intelligence, prompt-based interaction, REST APIs, web development, modular programming, and educational technology.

2. Problem Statement

Students often need quick explanations, revision material, practice questions, and guidance while studying. Traditional resources may require searching through several sources. EduGenie brings common learning-support functions into one lightweight application.

3. Objectives

- Provide quick question-and-answer support for academic topics.
- Explain difficult concepts in simple and readable language.
- Generate multiple-choice quizzes for active learning.
- Summarize long educational passages for revision.
- Create beginner-to-advanced learning paths and resource guidance.
- Demonstrate integration of an AI service with a FastAPI web application.

4. Scope of the Project

The project is suitable for students, self-learners and educational demonstrations. It focuses on five core learning functions: Q&A, explanation, quiz generation, summarization, and learning-path recommendations. The modular design allows additional functions to be added later.

5. Use-Case Scenarios

- Question answering: a learner enters an academic question and receives an AI-generated response.
- Concept explanation: a learner enters a difficult topic and requests a simple explanation.
- Quiz generation: a learner supplies a passage and receives multiple-choice questions.
- Summarization: a long passage is converted into a concise revision summary.
- Learning path: a learner enters a subject and receives a structured beginner-to-advanced study path.

6. System Architecture

EduGenie - System Architecture

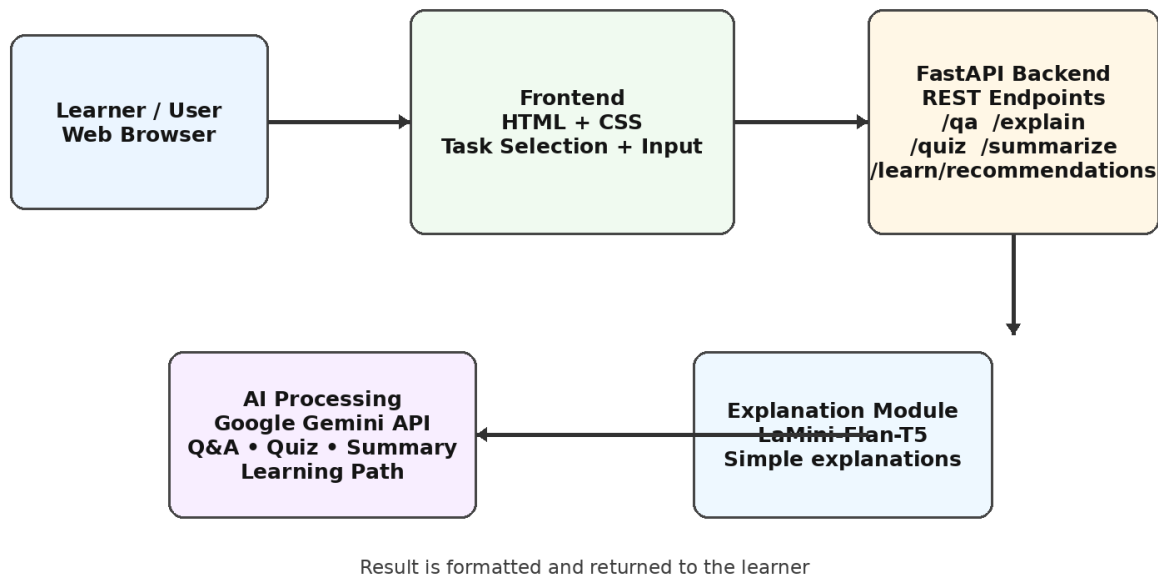


Figure 1. EduGenie high-level system architecture

7. Prerequisites and Technologies

Technology	Purpose
Python	Backend and AI integration
FastAPI	REST API framework
Google Gemini API	AI processing for learning tasks
HTML & CSS	Frontend and styling
Uvicorn	ASGI server
Jinja2	HTML templating support
Internet/API access	Required for cloud AI requests

8. Project Workflow

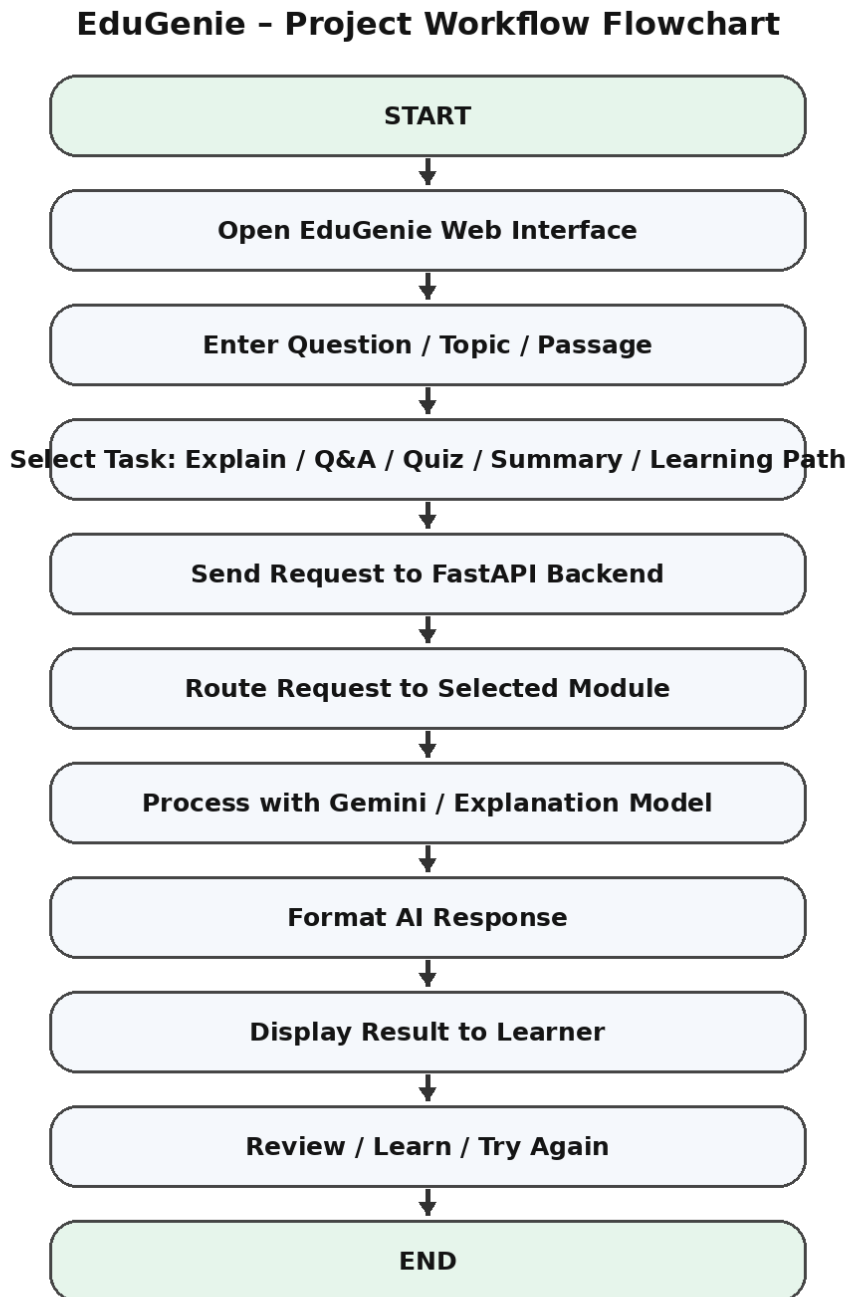


Figure 2. Main project workflow

Start → open interface → enter learning content → select a task → send request → FastAPI routes it → AI module processes it → result is displayed → learner reviews and continues.

9. Folder Architecture

```
EduGenie/  
├─ main.py  
├─ explanation_module.py  
├─ qna.py  
├─ quiz_module.py  
├─ summary_module.py  
├─ learning_path.py  
├─ templates/index.html  
├─ static/style.css  
└─ requirements.txt
```

10. Core Functional Modules

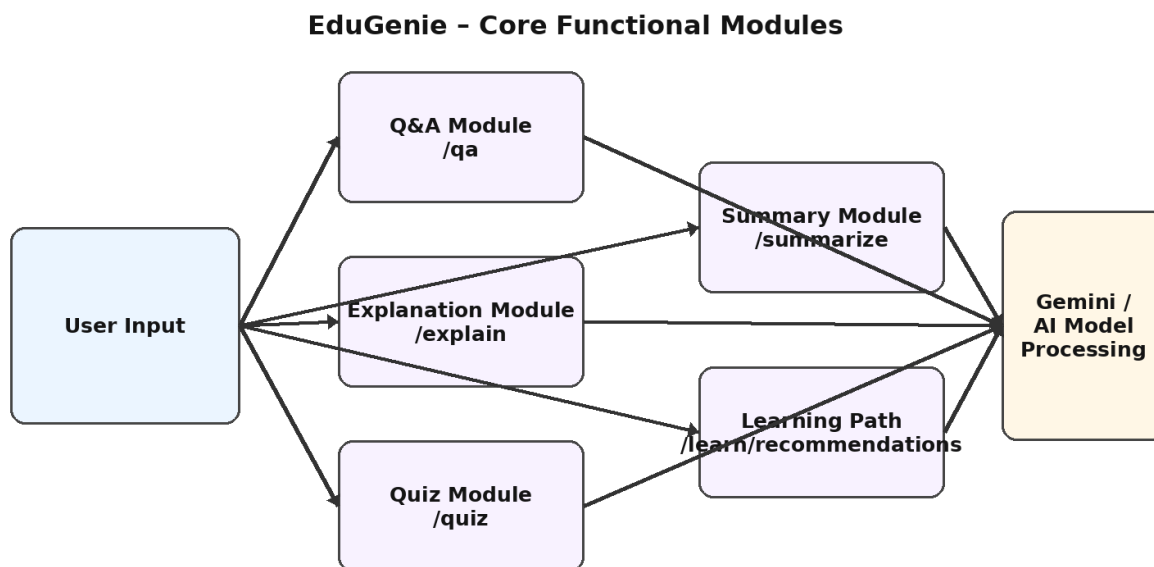


Figure 3. EduGenie functional modules

10.1 Q&A Module

Receives a question and sends an instruction to the AI model to produce a clear educational answer.

10.2 Explanation Module

Converts a difficult concept into simpler language. A lightweight local model such as LaMini-Flan-T5 can be used for this function.

10.3 Quiz Module

Generates three multiple-choice questions with four options each from supplied educational content.

10.4 Summary Module

Transforms a long passage into a concise summary while retaining important ideas.

10.5 Learning Path Module

Creates a structured learning plan from beginner to advanced level, including suggested topics and resources.

11. Backend API with FastAPI

The backend exposes separate endpoints so each learning function can be tested independently.

Endpoint	Purpose
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Text summarization
/learn/recommendations	Learning recommendations

12. Frontend Development

- Task dropdown: Explain, Q&A, Quiz, Summary and Recommend Path.
- Text area for the learner's question or educational passage.
- Submit button for sending the request.
- Result container for displaying the response.
- Responsive HTML/CSS layout for different screen sizes.

13. Coding Implementation

Example FastAPI backend using the Google GenAI Python SDK. Store the API key in an environment variable rather than directly in the source code.

```
# main.py
import os
from fastapi import FastAPI
from pydantic import BaseModel
from google import genai

app = FastAPI(title="EduGenie - Google Gemini Learning Assistant")
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
MODEL = "gemini-2.0-flash"

class RequestData(BaseModel):
    text: str

def ask_gemini(prompt: str) -> str:
    response = client.models.generate_content(
        model=MODEL,
        contents=prompt
    )
    return response.text
```

```

@app.post("/qa")
def question_answer(data: RequestData):
    return {"answer": ask_gemini(
        f"Answer this student question clearly: {data.text}")}

@app.post("/explain")
def explain(data: RequestData):
    return {"explanation": ask_gemini(
        f"Explain this topic simply for a student: {data.text}")}

@app.post("/summarize")
def summarize(data: RequestData):
    return {"summary": ask_gemini(
        f"Summarize this passage in clear bullet points: {data.text}")}

@app.post("/quiz")
def quiz(data: RequestData):
    prompt = ("Create 3 MCQs from this passage. Give 4 options and "
        "mark the correct answer.\n\n" + data.text)
    return {"quiz": ask_gemini(prompt)}

@app.post("/learn/recommendations")
def learning_path(data: RequestData):
    prompt = ("Create a beginner-to-advanced learning path with "
        "topics and practice suggestions:\n" + data.text)
    return {"learning_path": ask_gemini(prompt)}

@app.get("/")
def home():
    return {"message": "EduGenie is running"}

```

13.1 Requirements

```

fastapi
uvicorn
google-genai
pydantic

```

13.2 Run the Application

```

pip install -r requirements.txt
# Set GEMINI_API_KEY in the environment
uvicorn main:app --reload

```

Open: <http://127.0.0.1:8000>

14. Sample Input and Output

Module	Sample Input	Expected Output
Q&A	What is photosynthesis?	Clear AI-generated answer
Explanation	Explain database normalization	Simple explanation with examples
Quiz	Passage about computer networks	3 MCQs with 4 options each
Summary	Long educational paragraph	Concise revision summary

15. Functional Testing

1. Open the application and confirm the home endpoint responds.
2. Submit a question to /qa and verify an answer is returned.
3. Submit a concept to /explain and verify a simplified explanation.
4. Submit a passage to /quiz and verify three MCQs are generated.
5. Submit a passage to /summarize and verify a concise summary.
6. Submit a topic to /learn/recommendations and verify a staged learning path.

16. Importance of EduGenie

- Simplifies difficult educational concepts.
- Provides quick question-and-answer assistance.
- Supports active learning through quiz generation.
- Helps students revise long passages using summaries.
- Provides structured learning guidance from beginner to advanced levels.
- Demonstrates practical use of AI, APIs and web development.
- Modular architecture makes future expansion easier.

17. Advantages

- Single interface for multiple learning tasks.
- Fast response for common educational queries.
- Simple and student-friendly interaction.
- Can be extended with new modules.
- Can support different subjects and learning levels.

18. Limitations

- AI-generated responses should be checked against trusted sources when accuracy is important.
- Cloud-based AI functions require API access and may have usage limits.
- Response quality depends on the input and prompt.
- Sensitive personal information should not be entered into the application.

19. Future Enhancements

- Voice interaction
- Multilingual support
- Mobile application
- Progress dashboards and analytics
- Gamification
- Adaptive learning paths
- Group study and teacher/parent dashboards
- LMS integration

- Image/PDF input
- Smart notifications

20. Conclusion

EduGenie is an AI-powered educational assistant that combines a lightweight web interface, FastAPI backend, modular learning functions and generative AI. The project provides question answering, simple explanations, quizzes, summaries and personalized learning recommendations. It demonstrates how modern AI services can be integrated into an educational application while keeping the system modular and extendable.

21. Project Submission Details

Field	Details
University Name	Madras of university
College Name	Quiad-e-millath Government college for women (Autonomous)
College Code	1110005
Full Name	Mary.R
TN Skills Registered Email ID	ravimary593@gmail.com
Course Completion Email ID	ravimary593@gmail.com
NM ID	A60351E05E89EC1B2944EE728C205198
Project GitHub Repository	

PROJECT SUBMISSION

Submitted by: Mary.R
Date: 29/09/2026

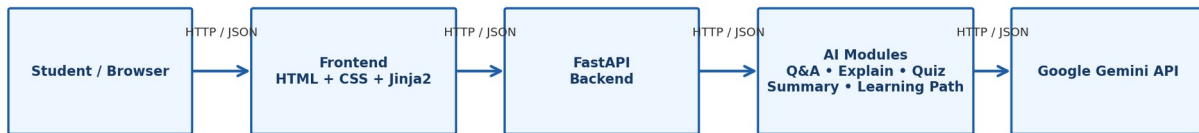
NAAN MUDHALVAN PROJECT

EduGenie: Google Gemini Powered Learning Assistant

Backend API with FastAPI & Web Interface

Generative AI Project

EduGenie - System Architecture



Optional data persistence can be added using a database layer.

University Name	Madras of university
College Name	Quiad-e-millath Government college for women (Autonomous)
College Code	1110005
Full Name	Sivasankari V
TN Skills Registered Email ID	sivasankari072008@gmail.com
Course Completion Email ID	sivasankari072008@gmail.com
NM ID	15EBC9B57B7BF349F87401753AD0B825
Project GitHub Repository	

Table of Contents

1. 1. Project Description
2. 2. Objectives
3. 3. Technologies Used
4. 4. Project Architecture
5. 5. Core Modules
6. 6. Backend API Endpoints
7. 7. Frontend Web Interface
8. 8. Project Workflow
9. 9. Milestones
10. 10. Prerequisites
11. 11. VS Code Setup
12. 12. Frontend UI Example
13. 13. Testing and Expected Results
14. 14. Conclusion

1. Project Description

EduGenie is a lightweight Generative AI powered educational assistant designed to simplify learning through natural-language interaction. The application uses Google Gemini for generating educational responses and FastAPI for providing a clean backend API. A simple HTML/CSS/Jinja2 web interface allows students to select a task, enter a question or learning material, and view the generated result.

The project focuses on five major learning features: question answering, topic explanation, quiz generation, text summarization, and personalized learning-path recommendations. The modular design makes the application easier to maintain, test, and extend.

2. Objectives

The main objectives of EduGenie are:

- Provide smart and concise answers to students' questions.
- Explain complex concepts in simple and readable language.
- Generate multiple-choice quizzes from a topic or passage.
- Summarize long educational passages for quick revision.
- Provide structured beginner-to-advanced learning recommendations.
- Create a simple and responsive web interface.
- Connect frontend requests to reusable FastAPI endpoints.
- Use secure environment-based configuration for the Gemini API key.

3. Technologies Used

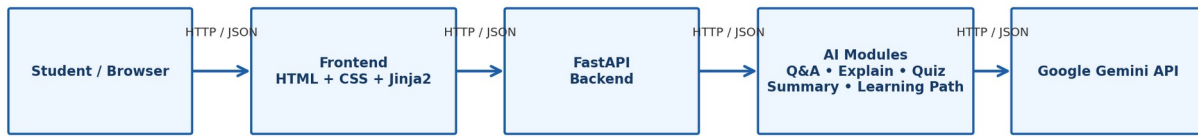
- Python 3.10+ – application programming language.
- FastAPI – backend web framework for building APIs.
- Uvicorn – ASGI server used to run the FastAPI application.
- HTML/CSS – frontend structure and styling.
- Jinja2 – server-side HTML templating.
- Google Gemini API – generative AI capabilities.
- python-dotenv – loading configuration such as API keys from an environment file.
- VS Code – development environment.
- JSON – request/response data exchange format.

4. Project Architecture

The system follows a modular client-server architecture. The student interacts with the frontend web page. The frontend sends an HTTP request to the FastAPI backend. FastAPI validates the request and routes it to the appropriate educational module. The module creates a suitable prompt and sends it to Google Gemini. The generated response is processed and returned to the frontend, where it is displayed to the student.

A database is optional in the basic version. If persistence is required later, a database layer can be added without changing the main frontend workflow.

EduGenie - System Architecture



Optional data persistence can be added using a database layer.

5. Core Modules

1. Explanation Module – explains educational topics in simple, context-aware language.
2. Q&A Module – answers general knowledge and academic questions using Gemini.
3. Quiz Module – generates multiple-choice questions with answer options and a suitable result format.
4. Summary Module – converts longer educational content into concise summaries while retaining key points.
5. Learning Path Module – creates a structured learning path from beginner to advanced level with suggested resources and steps.

6. Backend API Endpoints

The FastAPI backend exposes task-specific endpoints. Each endpoint accepts a request, validates the input, calls the relevant module, and returns a structured response.

POST /qa – question answering

POST /explain – topic explanation

POST /quiz – quiz generation

POST /summarize – text summarization

POST /learn/recommendations – personalized learning recommendations

The endpoints can exchange JSON data between the frontend and backend.

7. Frontend Web Interface

The frontend provides a simple interface for students. It contains a task selector, a text input area for a topic/question/passage, a submit button, and a result container. The interface is designed to be responsive and easy to use.

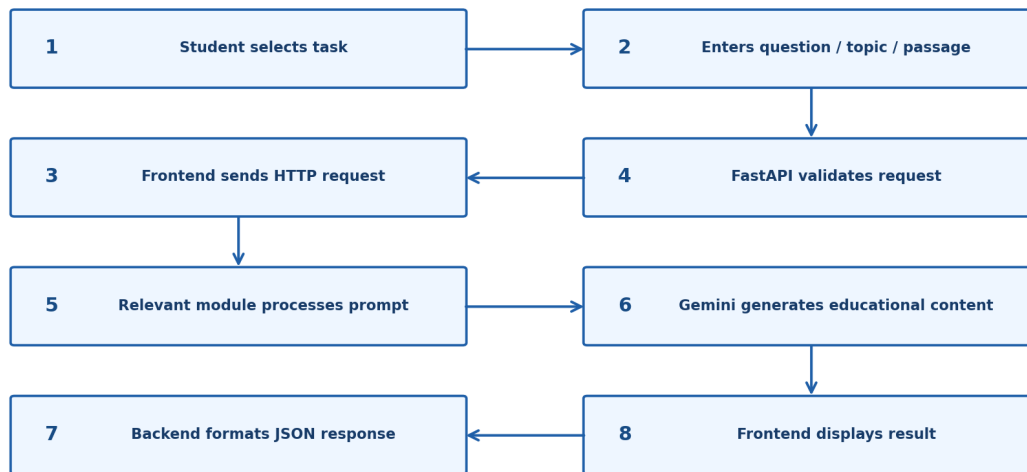
Typical user actions:

- Select Explain, Q&A, Quiz, Summary, or Recommend Path.
- Enter the required topic, question, or passage.
- Click Submit.
- Wait for the backend response.
- Read the generated educational result.

8. Project Workflow

1. Student selects an educational task.
2. Student enters a question, topic, or passage.
3. Frontend sends an HTTP request to the corresponding FastAPI endpoint.
4. FastAPI validates the request.
5. The backend calls the relevant educational module.
6. The module sends an appropriate prompt to Google Gemini.
7. Gemini returns generated educational content.
8. Backend processes the response and sends JSON to the frontend.
9. Frontend displays the result to the student.

EduGenie - Request Processing Flow



End - student receives the generated learning result

9. Milestones

Milestone 1 – Model Selection and Architecture

- Select Google Gemini for cloud-based generative AI.
- Design a modular architecture for maintenance and extension.

Milestone 2 – Core Functionalities Development

- Implement explanation, question answering, quiz, summarization, and personalized learning features.
- Connect each feature to a FastAPI endpoint.

Milestone 3 – Frontend Development

- Build the HTML form and task selector.
- Add text input and submit controls.
- Create responsive CSS styling.
- Connect frontend requests to FastAPI.
- Display results dynamically.

Milestone 4 – Testing and Deployment

- Run the application using Uvicorn.

- Test each endpoint and frontend workflow.
- Check topic explanation, quiz generation, summarization, and learning recommendations.

10. Prerequisites

- Python 3.10 or later
- VS Code or another Python IDE
- Internet connection for Gemini API
- Google account with access to the Gemini API
- Gemini API key
- Basic knowledge of Python, HTML, and REST APIs

11. VS Code Setup

1. Open VS Code and create/open the EduGenie project folder.
2. Open the VS Code terminal.
3. Create a virtual environment using: `python -m venv venv`
4. Activate the environment.
5. Install the required packages using: `pip install -r requirements.txt`
6. Create a `.env` file and add the Gemini API key.
7. Start the application using Uvicorn.
8. Open the local web page in a browser and test the available tasks.

12. Frontend UI Example

The user interface can contain the EduGenie title, a task dropdown, a large input area, a Submit button, and a result section. The dropdown can provide: Explain, Q&A, Quiz, Summary, and Recommend Path. The result section displays the generated content returned by the backend.

13. Testing and Expected Results

Testing should verify both API and user-interface behavior.

Test cases:

- Valid Q&A request → relevant educational answer.
- Valid explanation request → simple topic explanation.
- Valid quiz request → multiple-choice quiz content.
- Valid summary request → concise summary.
- Valid recommendation request → structured learning path.
- Empty input → validation message rather than an invalid API call.
- Invalid endpoint → appropriate HTTP error response.

Expected result: the student can select a learning task, submit educational content, and receive a readable Gemini-generated result through the web interface.

14. Conclusion

EduGenie demonstrates how a modern educational assistant can combine a FastAPI backend, a simple web interface, modular Python components, and Google Gemini generative AI. The project provides a foundation for interactive learning by supporting explanations, question answering, quizzes, summaries, and learning recommendations. Its modular architecture also makes future improvements such as authentication, database storage, analytics, and additional learning modules easier to implement.

Project File Structure

```
EduGenie/
|
|— main.py
|— explanation_module.py
|— qna.py
|— quiz_module.py
|— summary_module.py
|— learning_path.py
|— requirements.txt
|— .env
|— .env.example
|
|— templates/
|   |— index.html
|
|— static/
|   |— style.css
```

Student Details

University Name: Madras of university

College Name: Quiad-e-millath Government college for women (Autonomous)

College Code: 1110005

Full Name: Sivasankari V

TN Skills Registered Email ID: sivasankari072008@gmail.com

Course Completion Email ID: sivasankari072008@gmail.com

NM ID: 15EBC9B57B7BF349F87401753AD0B825

Project GitHub Repository: _____

Date: 29/09/2026

15. Complete Source Code

The following code provides a simple working implementation of the EduGenie prototype. Create the files using the project structure below. Add the Gemini API key only to the .env file and do not upload that secret key to GitHub.

15.1 requirements.txt

```
fastapi
uvicorn[standard]
jinja2
python-dotenv
google-generativeai
```

15.2 .env

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

15.3 .env.example

```
GEMINI_API_KEY=your_gemini_api_key_here
```

15.4 main.py

```
import os
from dotenv import load_dotenv
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel

from explanation_module import explain_topic
from qna import answer_question
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import recommend_learning_path

load_dotenv()

app = FastAPI(title="EduGenie API", version="1.0.0")
templates = Jinja2Templates(directory="templates")

class LearningRequest(BaseModel):
    text: str

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.post("/qa")
async def qa(data: LearningRequest):
    return {"result": answer_question(data.text)}
```

```

@app.post("/explain")
async def explain(data: LearningRequest):
    return {"result": explain_topic(data.text)}

@app.post("/quiz")
async def quiz(data: LearningRequest):
    return {"result": generate_quiz(data.text)}

@app.post("/summarize")
async def summarize(data: LearningRequest):
    return {"result": summarize_text(data.text)}

@app.post("/learn/recommendations")
async def recommendations(data: LearningRequest):
    return {"result": recommend_learning_path(data.text)}

```

15.5 explanation_module.py

```

import os
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")

def explain_topic(topic: str) -> str:
    prompt = f"""
    Explain the following educational topic for a college student.
    Use simple language, headings, examples and important points.
    Topic: {topic}
    """

    response = model.generate_content(prompt)
    return response.text

```

15.6 qna.py

```

import os
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")

def answer_question(question: str) -> str:
    prompt = f"""
    Answer this student's academic question accurately and clearly.
    Give a concise explanation and an example when useful.
    Question: {question}
    """

```

```
response = model.generate_content(prompt)
return response.text
```

15.7 quiz_module.py

```
import os
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")

def generate_quiz(topic: str) -> str:
    prompt = f"""
    Create 5 multiple-choice questions about the following topic.
    For each question provide four options (A, B, C, D) and identify the correct answer.
    Keep the questions suitable for a college student.
    Topic: {topic}
    """
    response = model.generate_content(prompt)
    return response.text
```

15.8 summary_module.py

```
import os
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")

def summarize_text(text: str) -> str:
    prompt = f"""
    Summarize the following educational passage.
    Keep the important facts and write the result as clear bullet points.
    Passage:
    {text}
    """
    response = model.generate_content(prompt)
    return response.text
```

15.9 learning_path.py

```
import os
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")

def recommend_learning_path(topic: str) -> str:
```

```

prompt = f"""
Create a beginner-to-advanced learning path for this topic.
Include stages, key concepts, practice activities and suggested next steps.
Topic: {topic}
"""

response = model.generate_content(prompt)
return response.text

```

15.10 templates/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduGenie - AI Learning Assistant</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <div class="container">
    <h1>&img alt="EduGenie logo" data-bbox="145 400 160 415"/> EduGenie</h1>
    <p class="subtitle">Your AI Learning Assistant</p>

    <div class="form-card">
      <label for="task">Select Task</label>
      <select id="task">
        <option value="explain">Explain</option>
        <option value="qa">Q&A</option>
        <option value="quiz">Quiz</option>
        <option value="summarize">Summary</option>
        <option value="recommendations">Recommend Path</option>
      </select>

      <label for="inputText">Enter topic, question or passage</label>
      <textarea id="inputText"
        placeholder="Type your question or learning content here..."></textarea>

      <button onclick="submitTask()">Submit</button>
    </div>

    <div class="result-card">
      <h2>Result</h2>
      <pre id="result">Your AI-generated result will appear here...</pre>
    </div>
  </div>

  <script>
    async function submitTask() {
      const task = document.getElementById("task").value;
      const text = document.getElementById("inputText").value.trim();
      const result = document.getElementById("result");

```

```

    if (!text) {
      result.textContent = "Please enter a topic, question or passage.";
      return;
    }

    result.textContent = "Generating response...";

    const endpoint = task === "recommendations"
      ? "/learn/recommendations"
      : "/" + task;

    try {
      const response = await fetch(endpoint, {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({text: text})
      });

      const data = await response.json();

      if (!response.ok) {
        result.textContent = data.detail || "Request failed.";
        return;
      }

      result.textContent = data.result;
    } catch (error) {
      result.textContent = "Unable to connect to the backend.";
    }
  }
</script>
</body>
</html>

```

15.11 static/style.css

```

body {
  font-family: Arial, sans-serif;
  background: #eef4fb;
  margin: 0;
  padding: 30px;
}

.container {
  max-width: 850px;
  margin: auto;
}

h1 {
  text-align: center;
  color: #174b84;
  margin-bottom: 5px;
}

```

```
}

.subtitle {
  text-align: center;
  color: #555;
}

.form-card, .result-card {
  background: white;
  padding: 24px;
  margin-top: 20px;
  border-radius: 12px;
  box-shadow: 0 3px 12px rgba(0,0,0,0.08);
}

label {
  display: block;
  margin-top: 10px;
  margin-bottom: 7px;
  font-weight: bold;
}

select, textarea {
  width: 100%;
  box-sizing: border-box;
  padding: 12px;
  border: 1px solid #b9c8d8;
  border-radius: 8px;
  font-size: 15px;
}

textarea {
  min-height: 170px;
  resize: vertical;
}

button {
  width: 100%;
  margin-top: 15px;
  padding: 12px;
  border: none;
  border-radius: 8px;
  background: #1f5fa8;
  color: white;
  font-size: 16px;
  cursor: pointer;
}

button:hover {
  background: #174b84;
}
```

```
pre {
  white-space: pre-wrap;
  line-height: 1.6;
}
```

15.12 Recommended Project Structure

```
EduGenie/
|
├── main.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── .env
├── .env.example
|
├── templates/
|   └── index.html
|
└── static/
    └── style.css
```

15.13 Run Commands

```
# Create virtual environment
python -m venv venv
```

```
# Windows
venv\Scripts\activate
```

```
# Install packages
pip install -r requirements.txt
```

```
# Run the FastAPI application
uvicorn main:app --reload
```

```
# Open in browser
http://127.0.0.1:8000
```

```
# FastAPI interactive API documentation
http://127.0.0.1:8000/docs
```

15.14 Sample API Request

```
POST /explain
Content-Type: application/json

{
  "text": "Explain the basics of microprocessors"
}
```

15.15 Sample API Response

```
{  
  "result": "A microprocessor is..."  
}
```

16. Coding Explanation

main.py is the central FastAPI application. It creates the web server, serves the frontend page, defines the API endpoints, validates incoming JSON using Pydantic, and connects each endpoint to a separate educational module. The module files keep the application organized by feature.

The frontend uses HTML and CSS for the interface and JavaScript fetch() to send HTTP POST requests to FastAPI. The backend returns JSON, and JavaScript places the returned result into the result area.

The Gemini API key is loaded from .env using python-dotenv. The .env file should remain private and should be excluded from version control.

17. GitHub Preparation

Before uploading the project to GitHub, create a .gitignore file and exclude the virtual environment and secret configuration.

17.1 .gitignore

```
venv/  
.env  
__pycache__/  
*.pyc
```

Project GitHub Repository: _____

18. Important Note

The Google Gemini SDK/API syntax can change over time. If the installed Gemini package uses a newer Google GenAI SDK, the model initialization and generation calls may need to be updated to that SDK's current syntax. The architecture and FastAPI endpoint design remain the same.

EduGenie: Google Gemini Powered Learning Assistant

Reference Project Board – Selected Activities

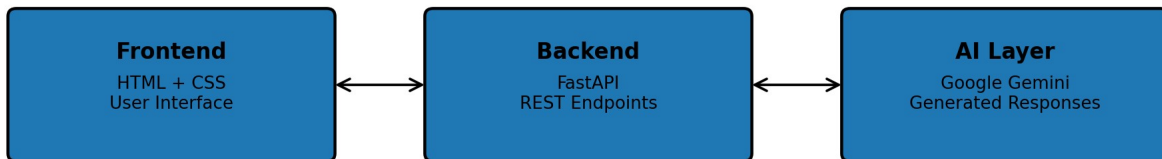
EduGenie: Google Gemini Powered Learning Assistant

EduGenie is a Generative AI based educational assistant designed to support learners through question answering, topic explanation, quiz generation, summarization and personalized learning recommendations.

Selected Activities

- Activity 3.2 – Live Integration
- Activity 4.1 – Run Locally

EduGenie - Basic Project Architecture



Tasks: Q&A | Explain | Quiz | Summary | Learning Path

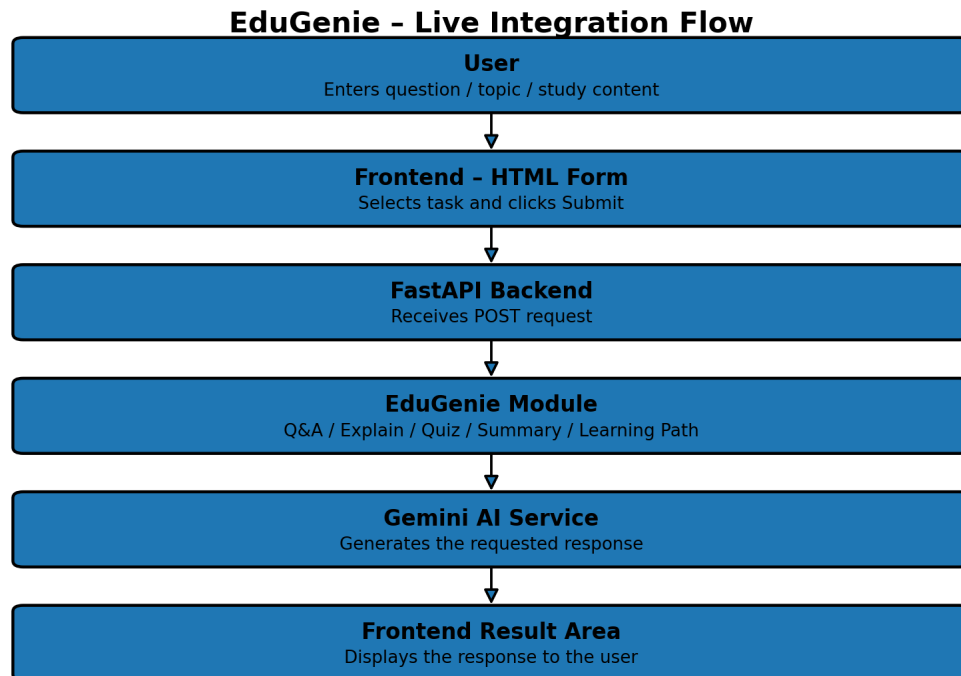
1. Activity 3.2 – Live Integration

Live Integration connects the EduGenie frontend with the FastAPI backend. The user selects a task, enters content and submits the request. The frontend sends a POST request to the backend, the relevant EduGenie module processes the request, and the generated response is displayed in the result area.

Objective

- Connect the HTML interface with the FastAPI backend.
- Send user input and the selected task to the correct endpoint.
- Receive the generated AI response.
- Display the result dynamically on the same interface.

Live Integration Flowchart



Frontend Components

- HTML form – user input and task selection.
- Task dropdown – Explain, QnA, Quiz, Summary and Recommend Path.
- Text area – accepts a question, topic or study passage.
- Submit button – sends the request.
- Result container – displays the generated response.
- CSS – provides responsive and clear styling.

Backend Endpoints

Endpoint	Purpose
/qa	Question and Answer
/explain	Explain a topic in simple language
/quiz	Generate multiple-choice questions
/summarize	Summarize long content
/learn/recommendations	Generate a structured learning path

Live Integration Steps

1. Open the EduGenie web interface.
2. Select the required task.
3. Enter the question, topic or study content.
4. Click Submit.
5. The frontend sends a POST request to the FastAPI backend.
6. The backend calls the corresponding EduGenie module.
7. The AI-generated response is returned.
8. The result is displayed below the input area.

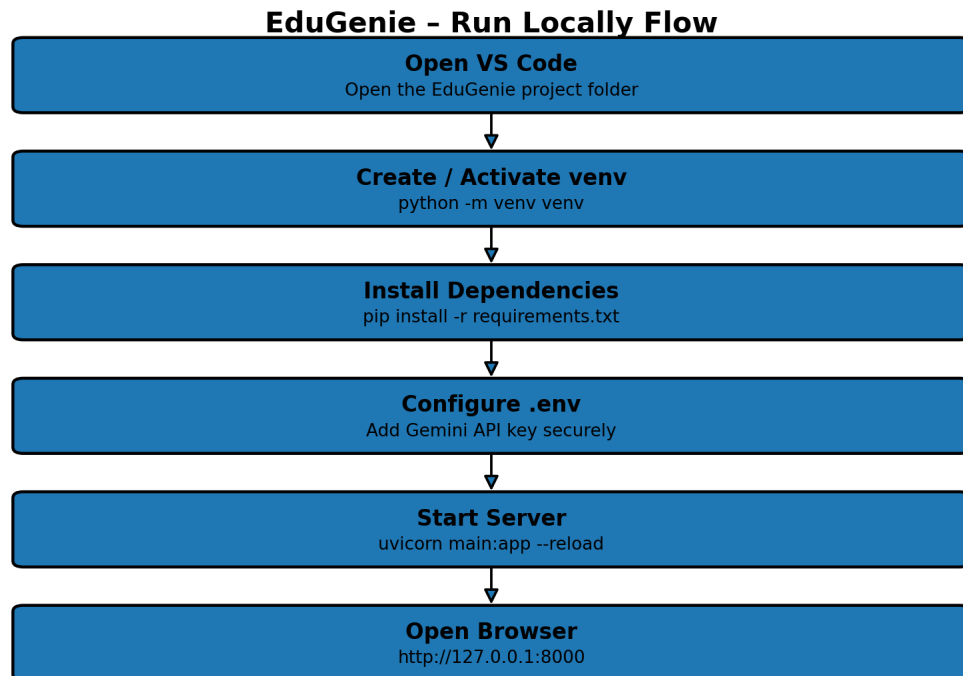
Expected Output

The frontend and backend should communicate successfully and the requested EduGenie response should appear in the result area without opening a separate page.

2. Activity 4.1 – Run Locally

Run Locally is the deployment activity used to start EduGenie on a local computer. The FastAPI application is launched using Uvicorn and opened through a local browser address.

Run Locally Flowchart



Prerequisites

- Python 3.10 or later
- Visual Studio Code
- EduGenie project folder
- Internet connection for Gemini API access
- Google account / Google AI Studio access
- Gemini API key
- Required packages listed in requirements.txt

VS Code Setup

9. Open VS Code and open the EduGenie project folder.
10. Open the VS Code Terminal.
11. Create a virtual environment: `python -m venv venv`
12. Activate the virtual environment.
13. Install dependencies: `pip install -r requirements.txt`
14. Create a `.env` file in the project root.
15. Add the Gemini API key using the environment variable expected by the application.

Start the Application

`uvicorn main:app --reload`

After the server starts, open the application in a browser using:

http://127.0.0.1:8000

Local Testing Checklist

- Check that Uvicorn starts without errors.
- Open the local browser address.
- Verify that the EduGenie interface loads.
- Submit a sample question.
- Verify that the response is displayed.
- Test explanation, quiz, summary and learning-path features when required.
- Stop the server using Ctrl + C after testing.

Security and Error Handling

- Keep the Gemini API key outside the source code by using an environment file.
- Do not upload the real .env file or API key to a public repository.
- Validate user input before sending requests.
- Handle API failures and invalid responses with clear error messages.

Expected Outcome

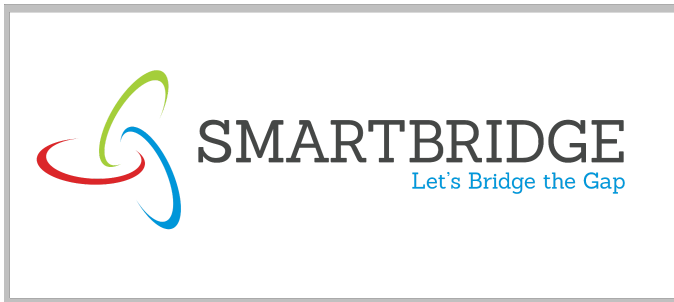
EduGenie should run successfully on the local computer and provide an interactive AI learning interface through the local

Submitted by

Name: Narmadha.R

Date: 29.09.2026

— End of Project Documentation —



EduGenie: Google Gemini Powered Learning Assistant

Project Description

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. Designed for students of all academic levels, EduGenie enables users to:

- Ask questions and receive smart, concise answers
- Understand complex concepts through simplified explanations
- Generate quizzes from topics or text
- Receive personalized learning recommendations
- Summarize large educational passages

Built with **FastAPI** for the backend and a simple **HTML+CSS frontend**, EduGenie leverages lightweight and cloud-based AI models for local efficiency and cloud power. It works well on devices like the Mac M1, making it accessible to a broad range of learners and developers.

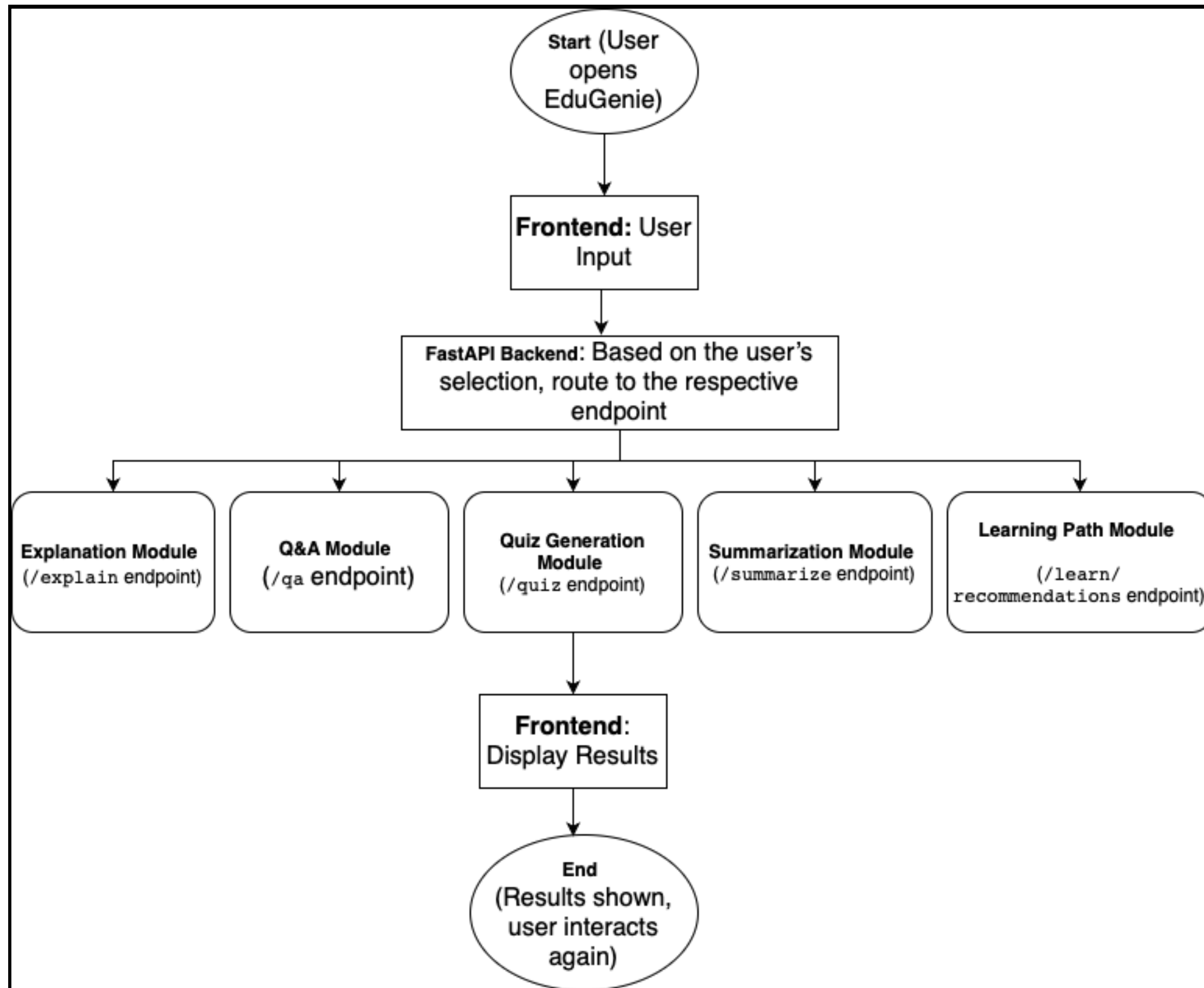
Scenarios

Scenario 1: A student wants to know about oceans and rivers uses EduGenie to ask “Which is the largest ocean?”

Scenario 2: A student wants to know the level of her understanding of “The Pythagoras Theorem” and clicks “Generate Quiz.”

Scenario 3: A learner exploring SQL requests a learning path which is a structured plan with beginner to advanced topics, timelines, and suggestions.

ARCHITECTURAL DIAGRAM



Pre-requisites

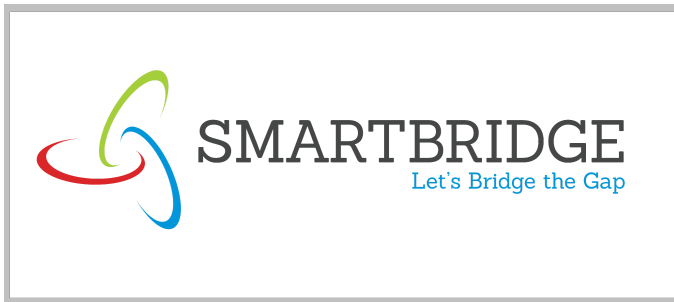
1. Python 3.10+

- Official Documentation: [Google AI for Developers](#)
- Installation Guide: [Tom's Hardware](#)
- Popular Tutorial: [Python Programming Tutorial - Full Course for Beginners](#)
- Installation Steps:

1. Download the latest Python 3.10+ installer for your operating system from the official website.
2. Run the installer and ensure you check the box that says "Add Python to PATH".
3. Follow the installation prompts to complete the setup.
4. Verify the installation by opening a terminal or command prompt and typing `python --version`.

2. FastAPI Framework –

- Official Documentation: [FastAPI](#)
- User Guide: [FastAPI](#)
- Popular Tutorial: [FastAPI Crash Course](#)



3. HTML & CSS – Basic templating used in /templates and /static

4. Google Gemini API Key

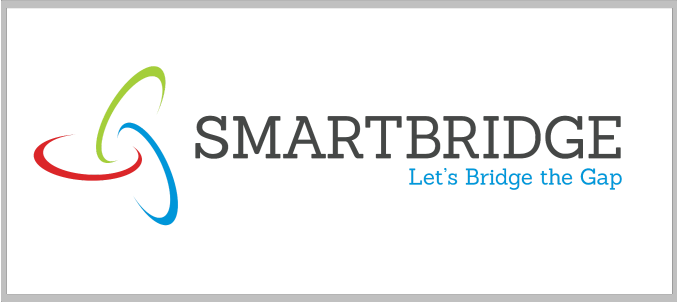
- Official Documentation: [Google AI for Developers](#)
- Setup Guide: [GeeksforGeeks](#)
- Popular Tutorial: [How to Use Google Gemini API Key](#)
- Setup Steps:
 1. Visit the Google AI Studio and sign in with your Google account.
 2. Create a new project and enable the Gemini API.
 3. Generate an API key and securely store it.

5. Uvicorn (ASGI Server)

- Official Documentation: [PyPI](#)
- Installation Steps:
 1. Install Uvicorn using pip: `pip install uvicorn`

6. Jinja2 (HTML Templating Engine)

- Official Documentation: [Jinja2 Documentation](#)
- Installation Steps:
 1. Install Jinja2 using pip: `pip install jinja2`



Project Workflow

MILESTONE 1: Model Selection and Architecture

Activity 1.1: Select AI Models

- **Gemini 1.5 Pro (via API):**
Used for: Q&A, summarization, quiz generation, learning paths
Benefits: Advanced reasoning, structured outputs, and cloud inference
- **LaMini-Flan-T5-783M (local):**
Used for: Concept explanation
Benefits: Instruction-tuned, lightweight, CPU-compatible

Folder Architecture:

EduGenie/		
1.	main.py	# FastAPI app
2.	explanation_module.py	# Concept explanation logic
3.	qna.py	# Question answering
4.	quiz_module.py	# Quiz generation
5.	summary_module.py	# Summarization
6.	learning_path.py	# Learning recommendations
7.	templates/index.html	# HTML frontend
8.	static/style.css	# Styling
9.	requirements.txt	# Python dependencies

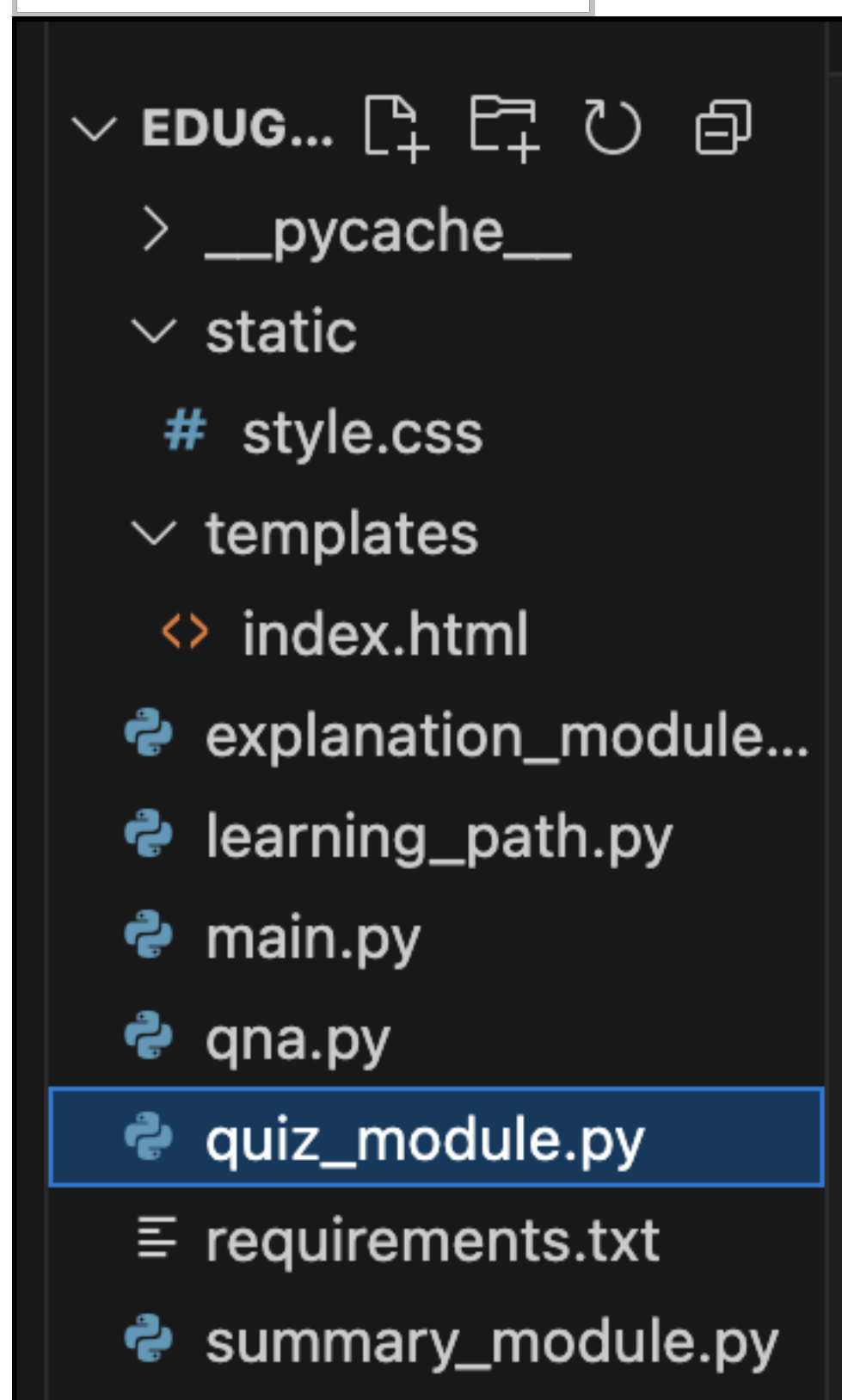


FIG. Showing the Folder Architecture

MILESTONE 2: Core Functionalities Development

Activity 2.1. : Module Implementation

1. Explanation Module

EduGenie utilizes the LaMini-Flan-T5 model, a lightweight yet powerful generative AI, to deliver educational content in a simplified and highly readable manner. This model is specifically fine-tuned to provide concise, context-aware responses that break down complex topics into easily understandable language. By focusing on clarity and brevity, EduGenie ensures that learners with minimal background knowledge or technical experience can grasp essential concepts without feeling overwhelmed. This makes it particularly valuable for beginners, school students, or self-learners seeking straightforward explanations. The integration of LaMini-Flan-T5 empowers EduGenie to act as a reliable and accessible study companion for foundational learning across subjects.

```

Explanation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        *inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
  
```

Fig. Explanation module logic

2. QnA Module

EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision. Its advanced understanding and contextual capabilities make it an effective tool for learners seeking accurate, AI-driven assistance across diverse educational topics and subjects.

```

def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠ Error in QnA: {e}"
  
```

Fig. Q/A Module

3. Quiz Module

EduGenie generates three multiple-choice questions (MCQs) from a given passage, each containing four carefully crafted options. It utilizes the Gemini model to comprehend the context and meaning of the passage, ensuring that the questions are relevant and the distractors are plausible. The final output is structured in JSON format for easy integration.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*?)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")

        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""

        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧹 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
```

Fig. Quiz Module Logic

It sends a structured prompt to the model, expecting a valid JSON response with each question containing four options and a correct answer. To ensure proper parsing, it cleans any Markdown code blocks using the `clean_json_block` function. The cleaned response is then parsed into a Python list. If an error occurs during generation or parsing, it returns a detailed error message for debugging.

4. Summary Module

This feature leverages Gemini's generative capabilities to summarize long paragraphs into concise, easy-to-understand versions, making it ideal for quick revision. It ensures the core information is retained while eliminating redundancy, helping learners absorb key points efficiently without losing clarity or context.

```
def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"⚠ Error in Summary: {e}"
```

Fig. Summary Module

5. Learning Path Module

The `get_learning_recommendations` function uses Google Gemini to generate a personalized, structured learning path for any given topic. By crafting a detailed prompt, it instructs the AI to suggest beginner to advanced concepts, organized by difficulty, and supported with useful resources such as videos, articles, or books. It ensures adaptability to the learner's level. The function safely handles API responses and errors, providing either the generated text or a helpful error message if the response is invalid or incomplete.

```
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (1
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🗨 Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc() # ✅ This gives full error trace
        return f"❌ Error occurred: {str(e)}"
```

Fig. Learning Path Module

Activity 2.2 : Backend API with FastAPI

- Defined RESTful endpoints for each module:

- /qa
 - /explain
 - /quiz
 - /summarize
 - /learn/recommendations
- Connected each endpoint to a respective module logic

```

main.py
40
41 # Q&A - GET API using Gemini
42 @app.get("/qa")
43 async def answer_question(question: str = Query(...)):
44     answer = answer_question_with_gemini(question)
45     return {"answer": answer}
46
47 # Explanation - POST API
48 @app.post("/explain/")
49 async def explain_api(request: Request):
50     data = await request.json()
51     topic = data.get("topic")
52     if not topic:
53         return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
54     explanation = explain_topic(topic)
55     return {"topic": topic, "explanation": explanation}
56
57 # Summarization - POST API
58 @app.post("/summarize/")
59 async def summarize_api(request: Request):
60     data = await request.json()
61     text = data.get("text")
62     if not text:
63         return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
64     summary = summarize_text(text)
65     return {"summary": summary}
66

```

Fig. Endpoints of API

```

66
67 # Quiz Generation - POST API
68 @app.post("/quiz")
69 async def quiz_api(request: Request):
70     data = await request.json()
71     text = data.get("text")
72     if not text:
73         return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
74     quiz = generate_quiz(text)
75     print("Generated quiz:", quiz) # ✅ DEBUG
76     return JSONResponse(content={"quiz": quiz})
77
78 # Learning Recommendations - GET API
79 @app.get("/learn/recommendations")
80 async def learning_recommendation_api(topic: str = Query(...)):
81     recommendation = get_learning_recommendations(topic)
82     return {"topic": topic, "recommendation": recommendation}
83
84

```

Fig. Endpoints of API

MILESTONE 3: Frontend Development

Activity 3.1: Build Web Interface

- HTML Form (index.html):
 - Task dropdown (Explain, QnA, Quiz, Summary, Recommend Path)
 - Text area for user input
 - Submit button
- CSS Styling (style.css):
 - Responsive design
 - Styled buttons, input areas, and result container

```

templates > <> index.html > ...
 2  <html lang="en">
 3  <head>
86  </head>
87  <body>
88    <h1>Welcome to EduGenie 🧙‍♂️🌟</h1>
89    <p>Your personal AI tutor for learning support!</p>
90
91    <!-- Q&A -->
92    <form id="qaForm">
93      <label><strong>Ask EduGenie a Question:</strong></label><br><br>
94      <input type="text" id="question" placeholder="Why is the sky blue?" required>
95      <button type="submit">Get Answer</button>
96    </form>
97    <div id="qaResult" class="output"></div>
98
99    <!-- Explanation -->
100   <form id="explainForm">
101     <label><strong>Need an Explanation?</strong></label><br><br>
102     <input type="text" id="topic" placeholder="Photosynthesis" required>
103     <button type="submit">Explain</button>
104   </form>
105   <div id="explanationResult" class="output"></div>
106
107   <!-- Summary -->
108   <form id="summaryForm">
109     <label><strong>Summarize a Paragraph:</strong></label><br><br>
110     <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
111     <button type="submit">Summarize</button>
112   </form>
113   <div id="summaryResult" class="output"></div>
114
115   <!-- Quiz -->
116   <form id="quizForm">
117     <label><strong>Generate a Quiz:</strong></label><br><br>
118     <input type="text" id="quizText" placeholder="Solar System" required>
119     <button type="submit">Generate Quiz</button>
120   </form>
121   <div id="quizResult" class="output"></div>

```

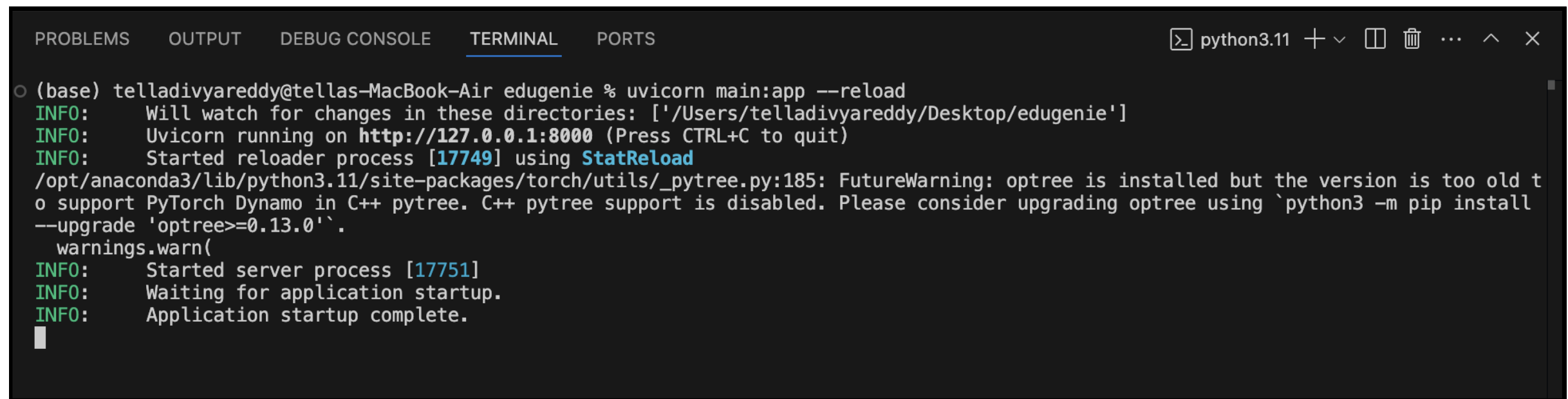
Fig. Showing Frontend Intergration

Activity 3.2: Live Integration

- Form submission sends POST request to FastAPI backend
- Result appears below input box in real-time

MILESTONE 4: Deployment

- **Activity 4.1: Run Locally**
Run locally with `uvicorn main:app --reload`
Navigate to: <http://127.0.0.1:8000>

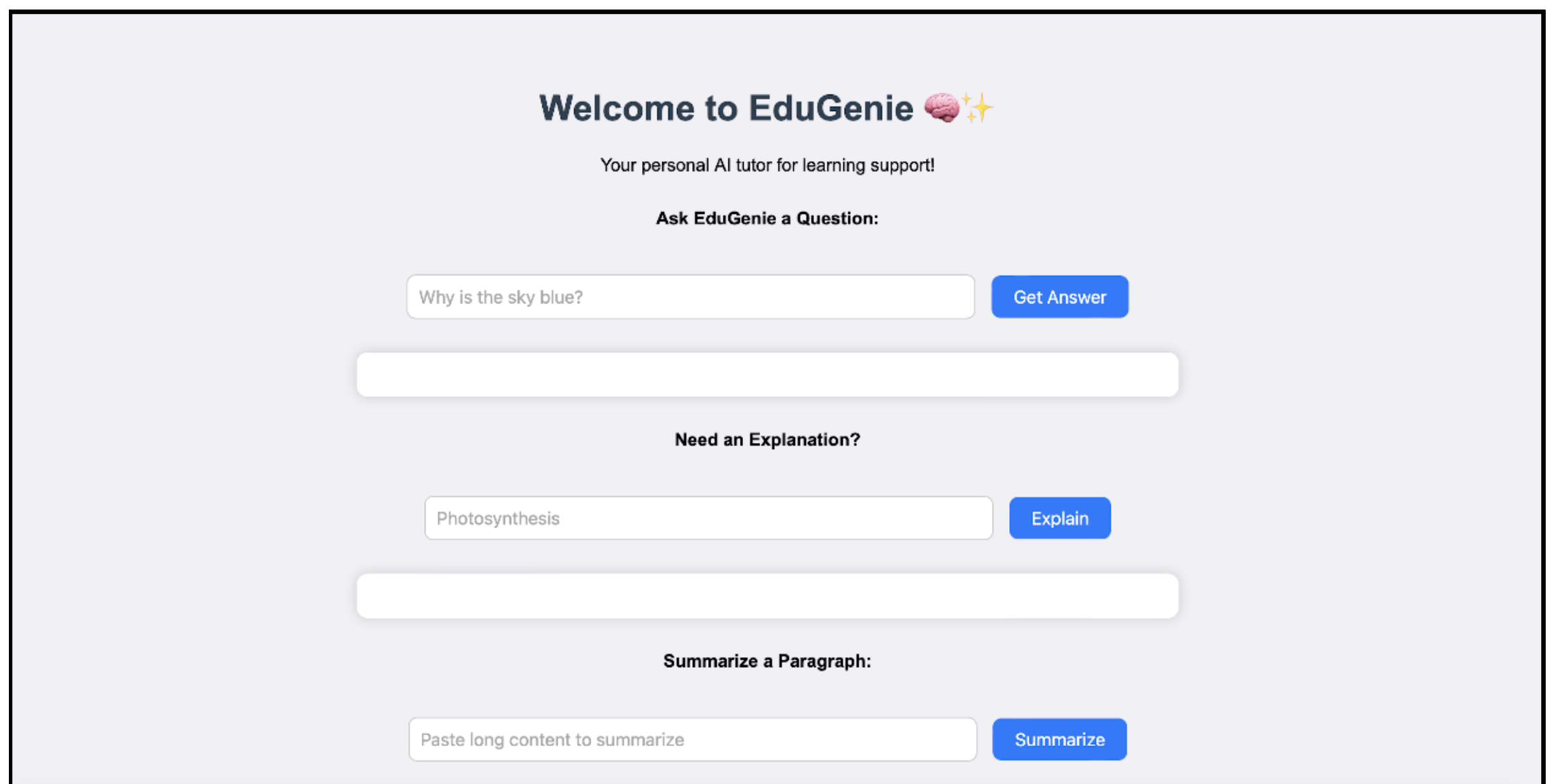


```

(base) telladivya@tellas-MacBook-Air edugenie % uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/Users/telladivya/Desktop/edugenie']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17749] using StatReload
/opt/anaconda3/lib/python3.11/site-packages/torch/utils/_pytree.py:185: FutureWarning: optree is installed but the version is too old to support PyTorch Dynamo in C++ pytree. C++ pytree support is disabled. Please consider upgrading optree using `python3 -m pip install --upgrade 'optree>=0.13.0'`.
  warnings.warn(
INFO: Started server process [17751]
INFO: Waiting for application startup.
INFO: Application startup complete.
  
```

Fig. Showing the Application can be accessed

- **Activity 4.2: Functional Testing**
Ask a question
Get an explanation
Generate a quiz
Summarize content
Get a personalized learning plan



Welcome to EduGenie 🧠✨

Your personal AI tutor for learning support!

Ask EduGenie a Question:

Need an Explanation?

Summarize a Paragraph:

Fig. EDUGENIE

Summarize a Paragraph:

Paste long content to summarize

Summarize

Generate a Quiz:

Solar System

Generate Quiz

Get Learning Recommendations:

e.g. Linear Regression

Get Recommendations

Fig. EDUGENIE

Exploring EduGenie:

a. Asking questions:

Welcome to EduGenie 🧠💡

Your personal AI tutor for learning support!

Ask EduGenie a Question:

which is the largest ocean?

Get Answer

Answer:

The Pacific Ocean is the largest ocean on Earth.

b. Explanation of any topic:

Need an Explanation?

quantum computing [Explain](#)

Explanation:

Quantum computing is a type of computing that uses the principles of quantum mechanics to perform certain calculations much faster than traditional computers. It allows computers to perform certain calculations much faster than traditional computers because it uses tiny particles called qubits that can exist in multiple states at the same time. This allows for new kinds of calculations and solving problems that are not possible with traditional computers.

Need an Explanation?

Binary Search Algorithm [Explain](#)

Explanation:

The binary search algorithm is a way to find the index of a target value in a sorted list of numbers. It's like searching for a needle in a haystack. You need to find the number between 0 and 1, and then go back to the beginning of the list to find the target value.

c. Summarising long paragraphs:

Summarize a Paragraph:

The Industrial Revolution, which began in the late 18th century, mark [Summarize](#)

Summary:

The Industrial Revolution changed the world by shifting it from farming to factories. New machines, like James Watt's steam engine, made things faster and easier to produce. While this boosted economies, it also created problems like bad working conditions, child labor, pollution, and overcrowded cities. Even with these problems, the Industrial Revolution shaped the modern world, affecting how we travel, communicate, work, and trade.

d. Generating quizzes:

It generates three questions with 4 options each. It corrects the answer if chosen wrong.

Generate a Quiz:

Pythagoras theorem

Generate Quiz

Quiz:

Q1: What does the Pythagorean theorem describe?

☐ The relationship between the angles of a triangle.

☐ The relationship between the sides of a right-angled triangle.

☐ The relationship between the area and perimeter of a triangle.

☐ The relationship between the sides of any triangle.

Check Answer

Quiz:

Q1: What does the Pythagorean theorem describe?

☐ The relationship between the angles of a triangle.

☒ The relationship between the sides of a right-angled triangle.

☐ The relationship between the area and perimeter of a triangle.

☐ The relationship between the sides of any triangle.

Check Answer

☒ Correct!

Q2: If 'a' and 'b' are the lengths of the two shorter sides of a right-angled triangle, and 'c' is the length of the longest side (hypotenuse), what equation represents the Pythagorean theorem?

☐ $a + b = c$

Q2: If 'a' and 'b' are the lengths of the two shorter sides of a right-angled triangle, and 'c' is the length of the longest side (hypotenuse), what equation represents the Pythagorean theorem?

☐ $a + b = c$

☐ $a^2 + b^2 = c^2$

☒ $a^2 - b^2 = c^2$

☐ $2a + 2b = 2c$

Check Answer

☒ Incorrect. Correct answer: $a^2 + b^2 = c^2$

Q3: Which type of triangle does the Pythagorean theorem apply to?

☐ Equilateral triangles

☐ Isosceles triangles

☒ Right-angled triangles

☐ All types of triangles

Check Answer

☒ Correct!

e. Guiding with learning recommendations from beginners level to advanced level

Get Learning Recommendations:

SQL

Get Recommendations

Learning Recommendations for "SQL":

SQL Learning Path: From Zero to Hero

This learning path is structured to progressively introduce SQL concepts, starting from the basics and gradually advancing to more complex topics. It's designed to be adaptive: feel free to adjust the pace and delve deeper into areas that particularly interest you.

I. Beginner Level: Building a Foundation

(Estimated Time: 1-2 weeks)

* **Key Topics:**

* What is a Database and SQL? (Relational Model, DBMS)

* Basic Syntax (SELECT, FROM, WHERE)

* Data Types (INT, VARCHAR, DATE, etc.)

* Filtering Data (WHERE clause with comparison operators, logical operators: AND, OR, NOT)

* Ordering Results (ORDER BY)

* Limiting Results (LIMIT/OFFSET or TOP)

* Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

complex topics. It's designed to be adaptive: feel free to adjust the pace and delve deeper into areas that particularly interest you.

I. Beginner Level: Building a Foundation

(Estimated Time: 1-2 weeks)

* **Key Topics:**

* What is a Database and SQL? (Relational Model, DBMS)

* Basic Syntax (SELECT, FROM, WHERE)

* Data Types (INT, VARCHAR, DATE, etc.)

* Filtering Data (WHERE clause with comparison operators, logical operators: AND, OR, NOT)

* Ordering Results (ORDER BY)

* Limiting Results (LIMIT/OFFSET or TOP)

* Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

* Introduction to Tables and Columns

* **Resources:**

* **Interactive Tutorials:**

* SQLZoo: Excellent for hands-on practice with different SQL flavors.

* Khan Academy's SQL course: Offers a good introductory overview.

* Codecademy's Learn SQL: Interactive lessons with practical exercises.

* **Videos:**

* FreeCodeCamp's SQL Tutorial: Comprehensive and beginner-friendly.

II. Intermediate Level: Working with Multiple Tables

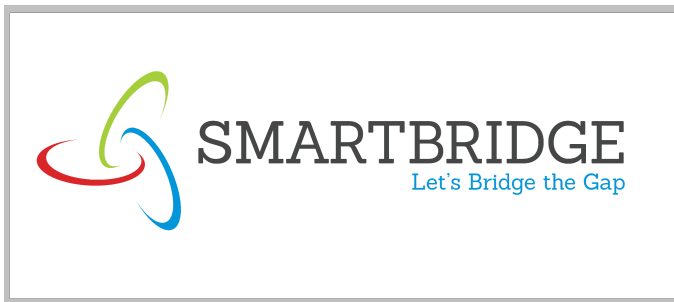
(Estimated Time: 2-3 weeks)

	II. Intermediate Level: Working with Multiple Tables **(Estimated Time: 2–3 weeks)** * **Key Topics:** * Joining Tables (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN) * Subqueries (Nested queries) * Grouping Data (GROUP BY) * Filtering Groups (HAVING) * Set Operations (UNION, INTERSECT, EXCEPT) * Working with Strings (string concatenation, substrings, LIKE operator) * Working with Dates and Times (date formatting, date arithmetic) * Views (Creating and using views) * **Resources:** * **Books:** * "SQL Queries for Mere Mortals" by Michael J. Hernandez: A great resource for understanding practical SQL querying. * **Online Documentation:** Refer to the documentation for your specific SQL database (e.g., MySQL, PostgreSQL, SQL Server). * **Practice Platforms:** LeetCode, HackerRank, and similar platforms offer SQL challenges to reinforce your skills. **III. Advanced Level: Mastering Database Management** **(Estimated Time: 3–4 weeks and beyond)** * **Key Topics:** * Stored Procedures and Functions * Triggers * Indexes and Performance Tuning * Transactions and Concurrency Control * Database Design (Normalization)	
	* **Books:** * "SQL Performance Explained" by Markus Winand: In-depth coverage of SQL performance optimization. * "Database Internals" by Alex Petrov: Explains the inner workings of database systems. * **Advanced Online Courses:** Look for specialized courses on database administration and performance tuning on platforms like Udemy and Coursera. * **Community Forums and Blogs:** Engage with the SQL community to learn from experienced professionals and stay up-to-date with the latest advancements. **Adaptive Learning Tips:** * **Start with the basics:** Don't rush into advanced topics before mastering the fundamentals. * **Practice regularly:** The more you practice writing SQL queries, the better you'll become. * **Use real-world datasets:** Working with real data will help you understand how SQL is used in practical scenarios. * **Focus on understanding the concepts:** Don't just memorize syntax; understand the underlying logic behind SQL queries. * **Break down complex problems:** Divide large problems into smaller, manageable parts. * **Don't be afraid to experiment:** Try different approaches and see what works best. * **Seek help when needed:** Don't hesitate to ask for assistance from online communities, forums, or mentors. This learning path provides a solid foundation for mastering SQL. Remember to adapt it to your own learning style and goals, and enjoy the journey of becoming proficient in this powerful language!	

It provides along with details of resources from where to learn and stepwise guidance throughout.

Conclusion:

EduGenie has been developed as a robust and accessible AI-powered educational assistant that seamlessly integrates cloud-based intelligence with an intuitive, low-footprint web interface. Designed to support self-learners, educational institutions, and content platforms, EduGenie democratizes learning by simplifying complex concepts, providing instant question-and-answer interactions, generating personalized quizzes, offering detailed



summaries, and tailoring smart learning paths based on individual needs. Its lightweight infrastructure ensures that even users with minimal hardware resources can access high-quality, personalized education without barriers.

Throughout the development of EduGenie, the project achieved key milestones in AI integration, user experience design, and content adaptability. Leveraging advanced generative AI, it transforms static learning into a dynamic and interactive process. The platform's simplicity and clarity cater to learners of all levels—especially those who may find traditional educational content overwhelming or inaccessible. The modularity of the platform further allows for easy future upgrades, while its cloud-based backend ensures scalability and data-driven intelligence.

However, the development journey came with its own set of challenges. Ensuring accuracy in AI-generated responses, maintaining performance across devices, and balancing personalization with general usability were significant hurdles. Overcoming these required continuous testing, fine-tuning AI prompt structures, optimizing the UI for clarity, and addressing data privacy concerns. Yet, these challenges provided valuable learning outcomes—especially in areas like prompt engineering, cloud integration, user-centric design, and scalable architecture.

Looking ahead, EduGenie holds immense potential for enhancement and expansion. One promising direction involves the integration of voice-based interaction, enabling users to learn hands-free using spoken commands and queries—a particularly useful feature for accessibility and multitasking. Multilingual support is another key frontier, helping bridge language barriers and reaching a broader global audience. A mobile application is also envisioned to make learning available anytime, anywhere, with offline features for uninterrupted access.

Furthermore, future iterations of EduGenie could incorporate progress tracking dashboards, allowing users to visualize their learning journey through metrics and insights. The addition of gamified elements such as badges, learning streaks, and challenge-based assessments can boost engagement and motivation. Adaptive learning paths powered by data analytics can guide users more intelligently through content based on their strengths and weaknesses.

From a collaboration perspective, EduGenie can evolve to support group study sessions, teacher or parent dashboards, and integration with Learning Management Systems (LMS) such as Moodle or Google Classroom. Input recognition for images and PDF files can expand how students interact with content, enabling snapshot-based doubt solving and resource summarization. Real-time sync and smart notifications will also improve continuity and interactivity.

In conclusion, EduGenie is not just an AI tool—it's a foundation for the future of personalized, inclusive, and intelligent education. By bridging gaps in access, comprehension, and adaptability, it redefines the learning experience for digital natives and underserved communities alike. As it continues to evolve, EduGenie is poised to become a comprehensive learning companion for the modern era—smart, scalable, and learner-first.



Submitted by:

Name:Vimala.V

Date:29.09.2026



NAAN MUDHALVAN PROJECT DOCUMENTATION

EduGenie: Google Gemini Powered Learning Assistant

AI-Powered Educational Assistant using FastAPI and a Web Interface

Project Documentation

1. Project Overview

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through generative AI. The system allows students to ask questions, understand complex concepts in simple language, generate quizzes, summarize educational passages, and receive personalized learning recommendations.

2. Problem Statement

Students often need quick explanations, revision material, quizzes, and structured learning guidance. Traditional resources may require searching through large amounts of content. EduGenie provides these learning functions through one simple web interface connected to AI services.

3. Objectives

- Provide concise answers to academic questions.
- Explain difficult concepts in simple and readable language.
- Generate multiple-choice quizzes from a topic or passage.
- Summarize long educational content for quick revision.
- Provide beginner-to-advanced learning recommendations.
- Offer a lightweight web application that can run on modest hardware.

4. Proposed System

The proposed system uses a FastAPI backend with separate modules for explanation, question answering, quiz generation, summarization, and learning recommendations. A simple HTML/CSS interface collects the learner's input and displays the generated result. The source documentation specifies Gemini for Q&A, quiz generation, summarization, and learning paths, and LaMini-Flan-T5 for concept explanation.

5. Technology Stack

Technology	Purpose
Python 3.10+	Application development
FastAPI	Backend REST API
Uvicorn	ASGI server
HTML + CSS	Frontend interface
Jinja2	HTML templating
Google Gemini API	Generative AI functions
LaMini-Flan-T5-783M	Local concept explanation

6. System Architecture

EduGenie High-Level Architecture

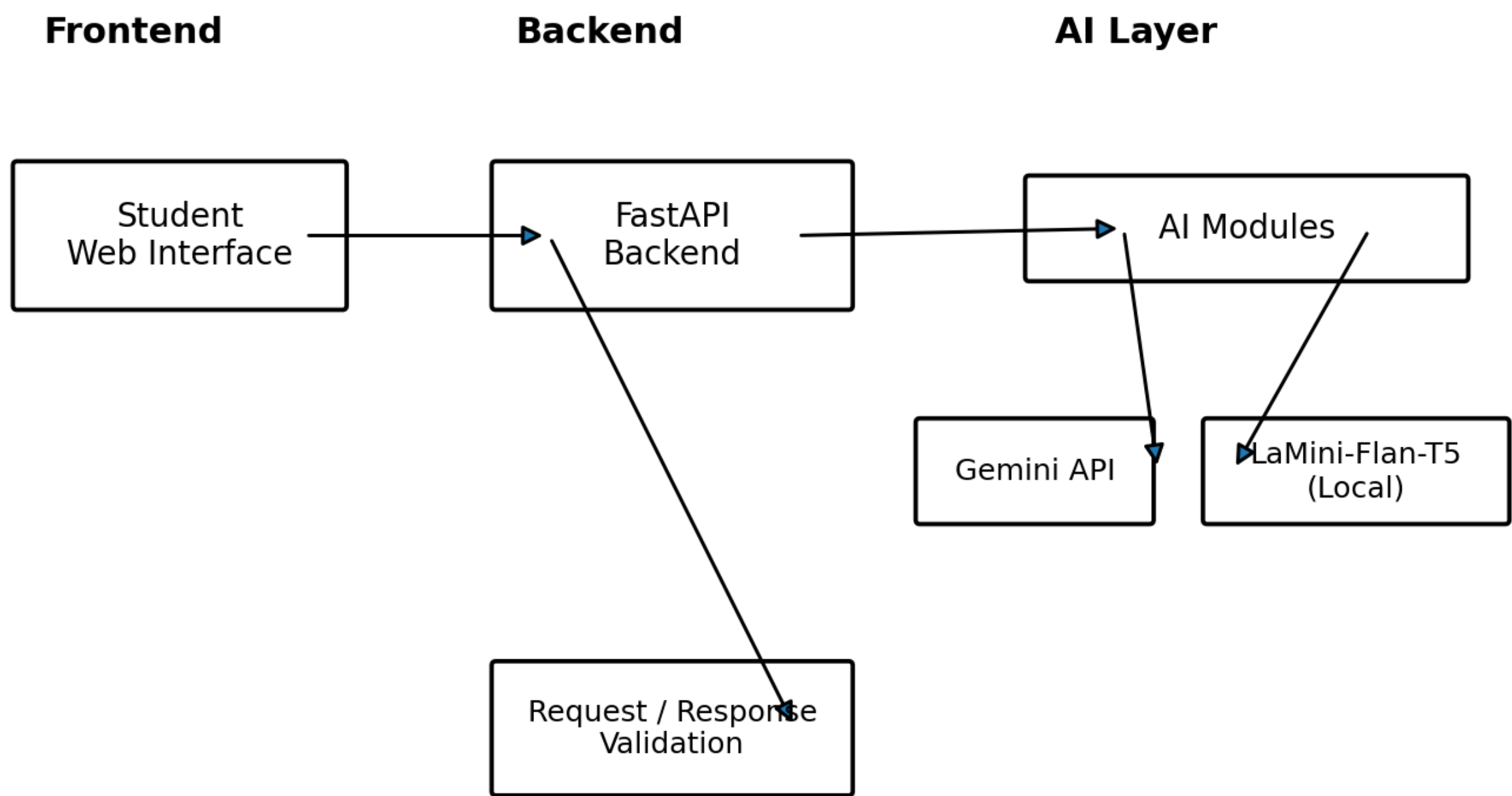


Figure 1. High-level architecture of EduGenie.

7. Folder Structure

The project documentation specifies the following core structure:

```
EduGenie/
├── main.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── templates/
│   └── index.html
├── static/
│   └── style.css
└── requirements.txt
```

8. Functional Modules

8.1 Explanation Module

Uses the lightweight LaMini-Flan-T5 model to provide concise, context-aware explanations that break complex concepts into easier language.

8.2 Question and Answer Module

Uses Gemini to answer general knowledge and academic questions through the /qa endpoint.

8.3 Quiz Module

Generates three multiple-choice questions with four options each. The expected output is structured as JSON so it can be integrated into the application.

8.4 Summary Module

Summarizes long paragraphs into concise, easy-to-understand content while retaining the main information.

8.5 Learning Path Module

Generates a structured learning path from beginner to advanced levels and can include learning resources such as videos, articles, or books.

9. API Endpoints

Endpoint	Function
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Text summarization
/learn/recommendations	Personalized learning recommendations

10. System Flowchart

EduGenie System Flowchart

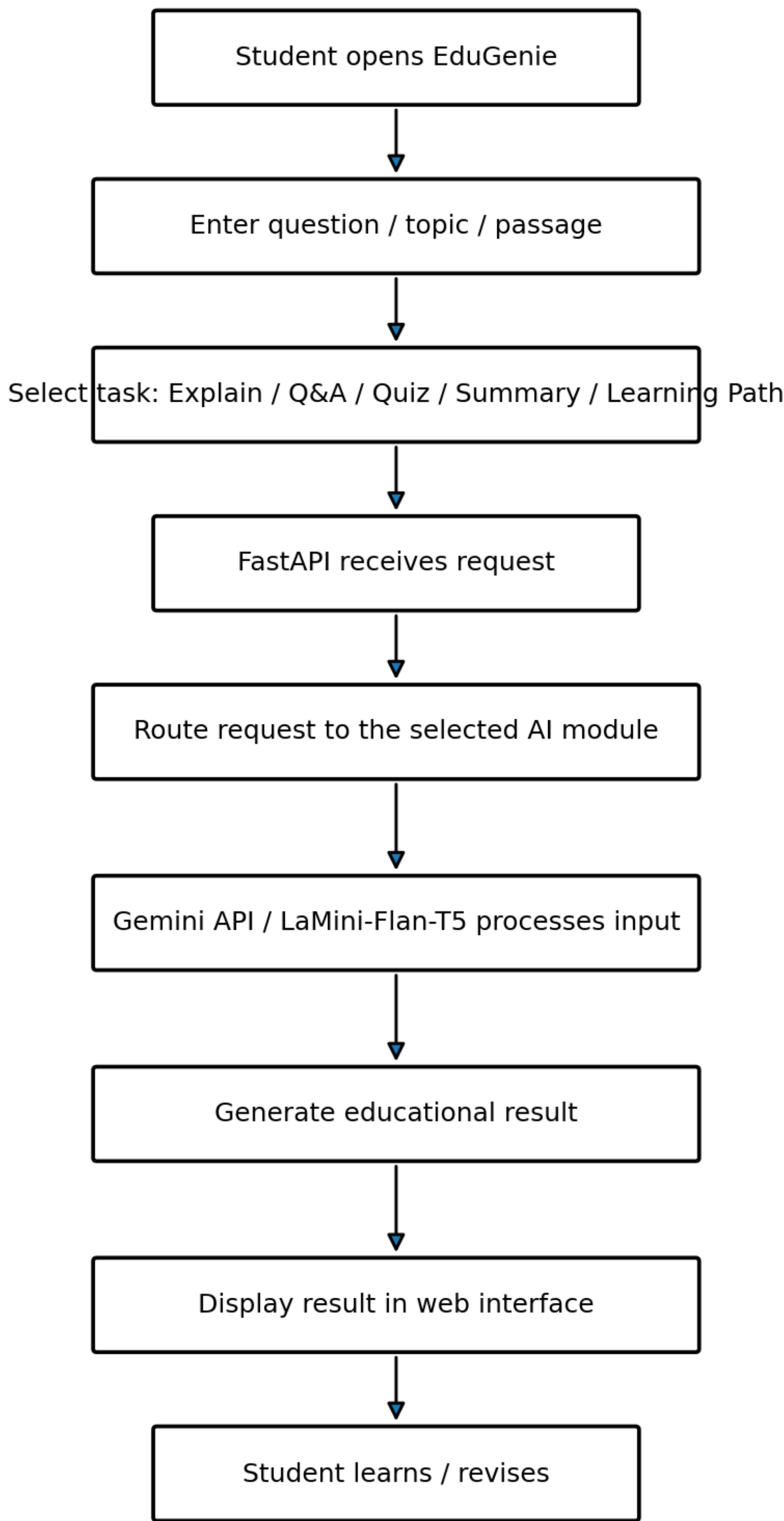
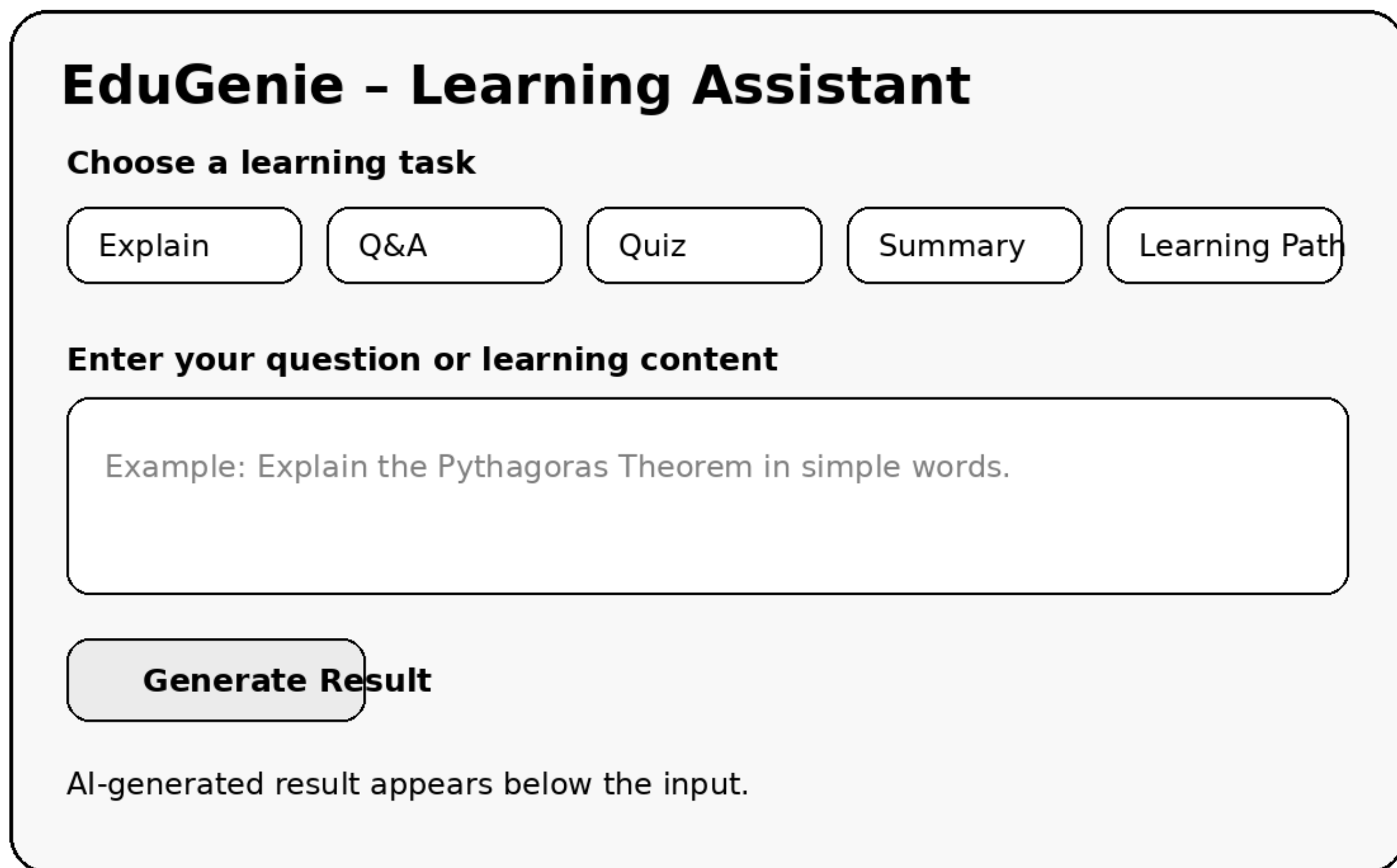


Figure 2. EduGenie end-to-end system flow.

11. Frontend Design

The frontend contains a task selection area, a text input area, a submit button, and a result section. The interface is designed to be responsive and simple for students.



The image shows a web interface for 'EduGenie - Learning Assistant'. It features a title, a task selection section with five buttons, a text input area with a placeholder example, a 'Generate Result' button, and a note about the AI-generated result.

EduGenie - Learning Assistant

Choose a learning task

Buttons: Explain, Q&A, Quiz, Summary, Learning Path

Enter your question or learning content

Example: Explain the Pythagoras Theorem in simple words.

Generate Result

AI-generated result appears below the input.

Figure 3. Illustrative EduGenie web interface.

12. Project Workflow / Milestones

- Milestone 1 – Model selection and architecture.
- Milestone 2 – Core functionality and FastAPI backend development.
- Milestone 3 – Web frontend development and live integration.
- Milestone 4 – Local deployment and functional testing.

13. Prerequisites and Installation

- Install Python 3.10 or newer and verify using `python --version`.
- Install the project dependencies from `requirements.txt`.
- Configure a Google Gemini API key securely.
- Install and use Uvicorn to run the FastAPI application.

The source documentation gives the local run command: `uvicorn main:app --reload`. After starting the server, open `http://127.0.0.1:8000` in a browser.

14. Functional Testing

Test Case	Input	Expected Result
TC01 – Q&A	Academic question	Concise answer is displayed
TC02 – Explanation	Complex topic	Simple explanation is displayed
TC03 – Quiz	Topic/passage	Three MCQs with four options are generated
TC04 – Summary	Long paragraph	Short summary is displayed
TC05 – Learning Path	Learning topic	Structured beginner-to-advanced path is displayed

15. Advantages

- Single interface for multiple learning tasks.
- Fast access to AI-generated educational assistance.
- Simple and lightweight web design.
- Modular backend makes individual features easier to maintain.
- Can support learners with different levels of prior knowledge.

16. Limitations

- AI-generated answers should be reviewed for accuracy.
- API availability and configuration affect cloud-based functions.
- Performance may vary depending on hardware and model usage.
- Internet access is required for cloud AI features described in the source project.

17. Future Enhancements

- Voice-based interaction.
- Multilingual support.
- Mobile application with offline capabilities.
- Learning progress dashboards.
- Gamification using badges, streaks, and challenges.
- Adaptive learning paths using learner data.
- Group study, teacher/parent dashboards, and LMS integration.
- Image and PDF input for doubt solving and summarization.
- Smart notifications and real-time synchronization.

18. Conclusion

EduGenie combines a simple web interface, FastAPI backend, and generative AI to create an accessible learning assistant. The project brings together question answering, concept explanation, quiz generation, summarization, and learning recommendations in one application. Its modular architecture also provides a foundation for future features such as voice interaction, multilingual learning, mobile access, progress tracking, and image/PDF-based learning.

19. Reference Project Information

This document is prepared specifically from the supplied EduGenie project documentation and is limited to the topic and functionality described there. The supplied source identifies the project as 'EduGenie: Google Gemini Powered Learning Assistant' and describes FastAPI, HTML/CSS, Gemini, LaMini-Flan-T5, the five functional endpoints, deployment, testing, conclusion, and future enhancements.

Submitted by

Name: Janani.A

Date:29.09.2026